COMPUTATIONALLY EFFICIENT STATIC RANSOMWARE DETECTION USING ENSEMBLE MACHINE LEARNING ON TABULAR PE DATA (A "GREEN AI" APPROACH)

BY

MORAH, DAVID NZUBECHUKWU

190805002

IN PARTIAL FULFILMENT OF THE AWARD OF MASTER OF SCIENCE IN COMPUTER SCIENCE, SUBMITTED TO THE COMPUTER SCIENCES DEPARTMENT, FACULTY OF COMPUTING AND INFORMATICS, UNIVERSITY OF LAGOS, AKOKA, YABA, LAGOS, NIGERIA

MARCH, 2026

CERTIFICATION

This is to certify that this research work was carried out by **MORAH, DAVID NZUBECHUKWU** with **Matric Number: 190805002**, a student of the Department of Computer Science, University of Lagos, Akoka, Yaba, and the same has been duly supervised and approved in partial fulfilment of the requirements for the award of Master of Science in Computer Science, University of Lagos State, Akoka, Yaba.

_____ _____

DR. N. A. AZEEZ **DATE**

Project *Supervisor*

_____ _____

DR. C. O. YINKA-BANJO **DATE**

Head of Department

DEDICATION

This work is dedicated to God, who has been my source of strength, wisdom and has provided all I need for this journey. I am grateful for the gift of life, the Grace of God, which has led me all through.

ACKNOWLEDGEMENT

Firstly, I want to thank God for the wisdom and strength to go through every stage of this journey. I would like to express my deep gratitude to my supervisor, Dr N. A. AZEEZ, for his consistent guidance, support, and encouragement throughout the course of this research. His vast expertise and mentorship were extremely valuable. To my parents, Mr and Mrs Morah, thank you for your unending support, prayers, and total belief in me. Your encouragement means more than words can express. To my elder sisters, Gozlyn and Sheryl Morah, thank you for being my biggest motivators. Your consistent push reminded me never to turn back, even in the most challenging moments. To all my colleagues and fellow project teammates who offered great support, thank you for the shared knowledge and support throughout this process.

TABLE OF CONTENTS

TITLE PAGE **1**
CERTIFICATION **2**
DEDICATION **3**
ACKNOWLEDGEMENT **4**
TABLE OF CONTENTS **5**
LIST OF TABLES **7**
LIST OF FIGURES **8**
ABSTRACT **9**
CHAPTER ONE **10**
INTRODUCTION **10**
1.1 Background To The Study 10
1.2 Problem Statement 14
1.3 Aim of the Study 15
1.4 Objectives of the Study 15
1.5 Justification for the Study 15
1.6 Scope of the Study 16
1.7 Methodology 17
1.8 Definition of Terms 18
1.9 Significance of the Study 19
CHAPTER TWO **20**
LITERATURE REVIEW **20**
2.1 Conceptual Framework of Machine Learning-Based Ransomware Detection 20
Ransomware 20
2.2 Static Analysis and PE Header-Based Detection Methods 23
2.3 Deep Learning Approaches for Ransomware Detection via PE Visualisation 24
2.4 Traditional and Ensemble Machine Learning Approaches 25
2.5 Concept Drift and Temporal Robustness in Malware Classification 27
2.6 Computational Efficiency and the Green AI Imperative in Cybersecurity 27
2.7 Challenges and Research Gaps 28
2.8 Summary of Reviewed Literature 29
2.9 The PE Header as a Feature Source for Ransomware Detection 35
2.10 Limitations of Deep Learning for Endpoint Ransomware Detection 35
CHAPTER THREE **37**
RESEARCH METHODOLOGY **37**
3.1 Introduction 37
3.2 Research Design 38
3.2.1 Proposed Ransomware Detection System Algorithm 39
3.3 Dataset Description 40
3.5 Machine Learning Algorithms 44
3.5.1 Gradient Boosting Family 45

3.5.2 Bagging Ensemble Family 46
3.5.3 Base Non-Linear Family 47
3.5.4 Linear/Probabilistic Family 48
3.6 Evaluation Metrics and Performance Analysis 48
3.6.1 Classification Performance Metrics 49
3.6.2 Computational Efficiency Metric 50
3.7 Hardware Environment and Green AI Validation 50
3.8 Comparative Analysis with Existing Techniques 51
3.9 Summary of Methodology 52
REFERENCES **53**

LIST OF TABLES

Page

Table 1: Summary of Reviewed Literature 29

Table 2: Summary of Datasets to be Used for the Study 40

Table 3: Summary of Machine Learning Algorithms to be Benchmarked 44

Table 4: Confusion Matrix Interpretation for Binary Ransomware Classification 49

Table 5: Deep Learning Baselines for Comparative Analysis 51

LIST OF FIGURES

Page

Figure 1.1: Global Ransomware Revenue, 2017-2024 11

Figure 1.2: Structure of the Windows Portable Executable (PE) File Format 12

Figure 2.1: Taxonomy of Malware Analysis Approaches for Ransomware Detection 21

Figure 3.1: Overview of the Research Design for Computationally Efficient Ransomware Detection 38

Figure 3.2: Data Preprocessing Pipeline for PE Header Datasets 42

Figure 3.3: Data Splitting Strategy for Model Training and Evaluation 44

Figure 3.4: Algorithm Taxonomy Across Four Mathematical Families 44

ABSTRACT

Ransomware is one of the most critical and rising challenges in the modern cybersecurity ecosystem that has caused irreversible data loss and financial damage across all industries. In this context, state-of-the-art ransomware detection approaches using PE header analysis are primarily based on deep learning techniques that utilize Convolutional Neural Networks to classify ransomware and benign executables by transforming PE header information into image tensors. Although these approaches have shown high classification accuracy, they are computationally expensive and require dedicated GPU support, high inference latency, and are considered black boxes. Therefore, this research will propose a computationally efficient solution by benchmarking ten different machine learning algorithms from four different mathematical backgrounds—Gradient Boosting (XGBoost, LightGBM, CatBoost), Bagging Ensemble (Random Forest, Extra Trees), Base Non-Linear (Decision Tree, SVM, KNN), and Linear/Probabilistic (Logistic Regression, Gaussian Naïve Bayes)—that will be trained and tested using structurally independent datasets to prove their applicability and robustness against concept drift. In this research, all ten different machine learning algorithms will be trained and tested using four different datasets to prove their applicability and robustness against concept drift. Performance evaluation will be conducted using eleven different metrics that include Accuracy, Precision, Recall, F1-Score, Matthews Correlation Coefficient, AUC ROC Curve, and training latency. All experiments will be conducted using a standard workstation that is commonly found at most endpoints and is not equipped with a GPU. The research is expected to show that ransomware detection using ensemble-based gradient boosting classifiers can achieve equal or even higher classification accuracy than deep learning-based approaches using orders-of-magnitude lower computational power and complexity and validate the Green AI concept in the ransomware detection ecosystem.

CHAPTER ONE
INTRODUCTION

1.1 Background To The Study

In the current cybersecurity environment, ransomware has become one of the most harmful and financially devastating types of malicious software. Over the past ten years, ransomware attacks have increased in frequency, sophistication, and financial impact. They differ from traditional malware in that their sole goal is to encrypt victim data and demand payment, usually in cryptocurrency, for a decryption key (Beaman et al., 2021). Ransomware is particularly dangerous due to the irreversible nature of cryptographic file encryption; unlike other types of malware that impair system functionality or steal data, crypto-ransomware permanently eliminates access to an organization's most valuable digital assets unless a legitimate decryption key is obtained (Kapoor et al., 2022). Because of this feature, ransomware is now the weapon of choice for both state-sponsored threat actors and financially motivated cybercriminals.

The threat posed by ransomware has significantly increased in recent years. The widespread COVID-19 pandemic was a major trigger because the quick transition to remote work required workers to connect to company networks from inadequately secured home environments, increasing the attack surface accessible to adversaries (Oz et al., 2022). During this time, healthcare organizations were disproportionately targeted because the industry's quick digitization revealed serious security flaws that hackers quickly exploited (Beaman et al., 2021). Beyond the healthcare industry, ransomware attacks on government agencies, educational institutions, and critical infrastructure have resulted in annual global damages estimated in the billions of dollars, and projections show that these damages will continue to rise (Meland et al., 2020). The year-over-year escalation of the financial impact of global ransomware attacks is illustrated in Figure 1.1.

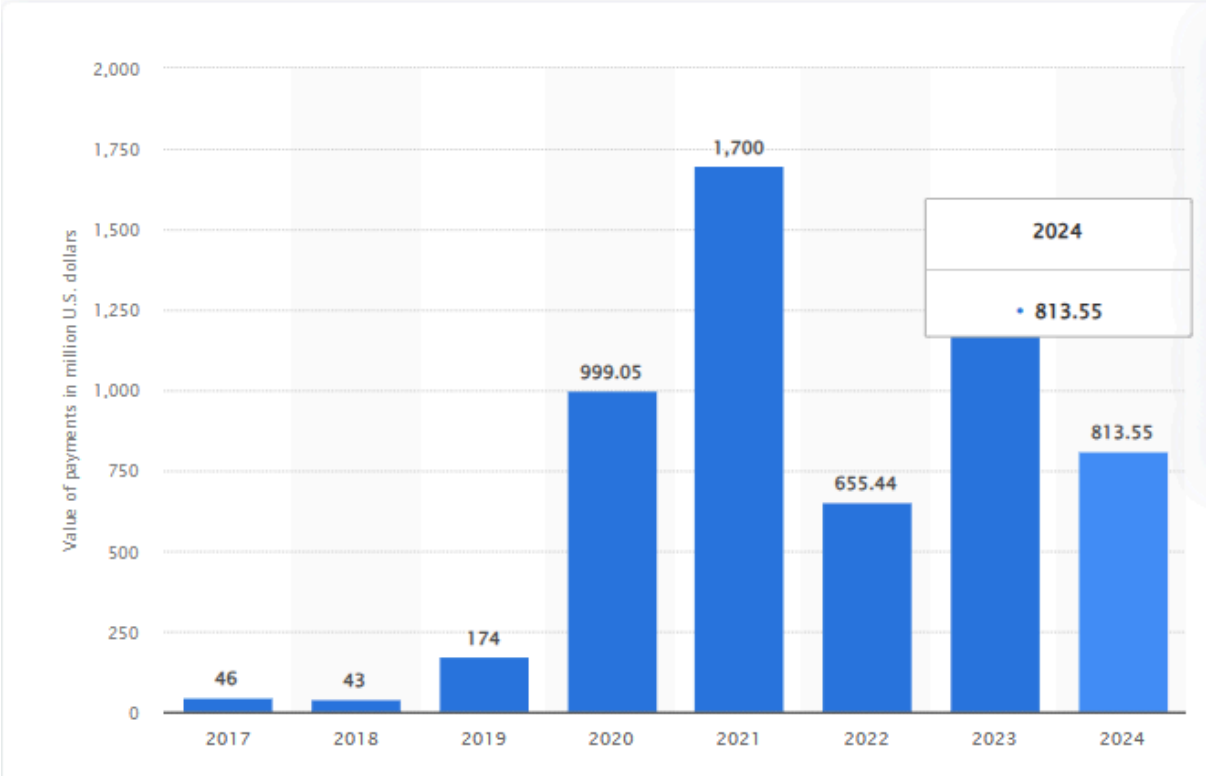


Figure 1.1: Total Annual Amount of Money Received by Ransomware Actors Worldwide, 2017–2024 (Source: Statista, 2025)

The threat of ransomware has been further democratized by the rise of Ransomware-as-a-Service (RaaS) platforms on darknet marketplaces, which allow even non-technical malevolent actors to use customized ransomware variants created from a shared code base (Meland et al., 2020). This franchise model has created a cooperative criminal ecosystem in which affiliates carry out attacks in return for a portion of the ransom money, while developers develop and maintain ransomware toolkits. As a result, new ransomware families and variants continue to proliferate, all of which are designed to circumvent modern detection systems and improve encryption effectiveness (Kok et al., 2022).

Ransomware defense strategies can be broadly classified into two analytical paradigms: dynamic analysis and static analysis. A suspicious executable is run in an isolated sandbox environment during dynamic analysis, and its runtime behavior—such as file system modifications, network communications, and cryptographic API invocations—is observed to ascertain whether it is malicious (Oz et al., 2022). Although this method has strong detection capabilities and is naturally more resistant to evasion strategies, it has serious operational drawbacks: It requires significant hardware resources, is computationally slow, and poses a

safety risk in the event that containment fails (Fernando et al., 2020). In contrast, static analysis extracts features like Portable Executable (PE) header metadata, opcode sequences, byte n-grams, string signatures, and Import Address Table (IAT) entries by analyzing an executable's structural characteristics without running it (Ucci et al., 2019). The main benefit of static analysis is its speed and safety, as detection can occur before program execution, enabling preemptive defense (Moreira et al., 2023).

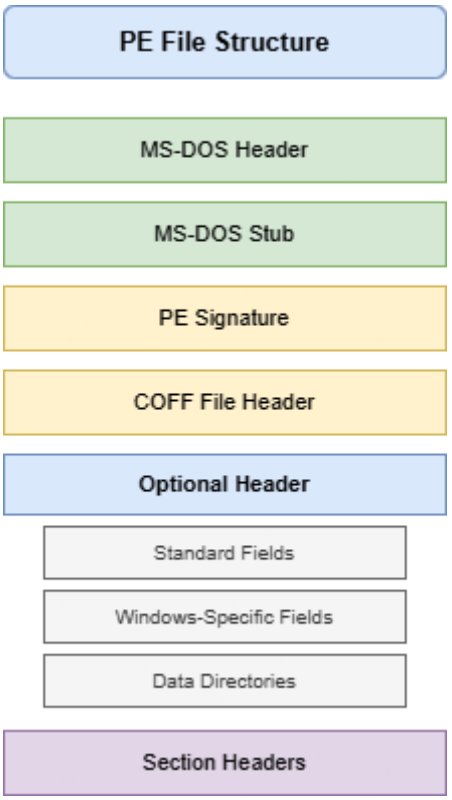


Figure 1.2: Structure of the Windows Portable Executable (PE) File Format (Adapted from Microsoft PE Format Specification)

The Microsoft Windows operating system's standard binary format for executables, Dynamic-Link Libraries (DLLs), and object code is the Portable Executable (PE) file format. The Machine Type, TimeDateStamp, SizeOfCode, AddressOfEntryPoint, SectionAlignment, and Data Directory entries are among the rich metadata found in its header structure that offer significant indicators of a binary's nature and intent (Moreira et al., 2023). As a structurally unique class of malware, ransomware displays distinctive subroutines that involve file enumeration, cryptographic API invocation, system information retrieval, and Command-and-Control (C2C) server communication. These subroutines all leave visible artifacts in PE header fields (Hampton et al., 2018; Kapoor et al., 2022). As a result, one of

the most popular feature extraction techniques for ransomware detection is PE header analysis.

Recent state-of-the-art (SOTA) works have increasingly used Deep Learning (DL) models, especially Convolutional Neural Networks (CNNs), to classify ransomware based on PE header features. For example, Moreira et al. (2023) proposed the Xception ColSeq approach, where the first 1,024 bytes of the PE header are converted into colour images arranged in a sequential vector pattern. These images are then classified using the Xception CNN architecture without transfer learning. The proposed approach achieves an accuracy of 98.20% on a dataset consisting of 25 ransomware families. Similarly, Manavi and Hamzeh (2021) used Long Short-Term Memory (LSTM) networks on PE header byte sequences, achieving 93.25% accuracy. Furthermore, the authors proposed another approach using CNNs on grey-scale zigzag-patterned images of PE headers, achieving 93.33% accuracy (Manavi & Hamzeh, 2022). Although these approaches have shown promising discriminative power, these models are associated with high computational requirements, including hardware needs, training time, and energy consumption, which are barriers to their deployment on regular workstations (Schwartz et al., 2020).

This goes against the emerging need for sustainable and computationally efficient artificial intelligence, also referred to as "Green AI" (Schwartz et al., 2020). The environmental and financial costs of computationally intensive AI approaches have been well documented, with research showing that the carbon footprint of training a single large neural network model can equal that of five automobiles (Strubell et al., 2019). In the context of endpoint ransomware detection, the requirement for computationally efficient approaches and the impracticality of using heavyweight models like CNNs for organizations that are not able to leverage cloud infrastructure are also evident.

The present study will thus propose a new paradigm for PE classification based on computationally efficient ensemble-based machine learning classifiers that operate directly on structured tabular PE header information, thus avoiding the computationally costly image tensor conversion process altogether. By benchmarking ten different algorithms across four structurally different data sets, this study will show that lightweight classifiers, especially ensemble-based gradient-boosting classifiers such as XGBoost, LightGBM, and CatBoost, can deliver classification results at least on par with their deep learning counterparts while being orders of magnitude more computationally efficient and easier to interpret.

1.2 Problem Statement

The current state of the research for static ransomware detection via PE header-based approaches is dominated by computationally intensive Deep Learning-based solutions, especially the use of Convolutional Neural Networks (CNN), which necessitate the conversion of the raw PE header byte sequences to two-dimensional image tensors for the classification process (Moreira et al., 2023; Manavi & Hamzeh, 2022). Even though the above approaches have shown promising results in the detection of ransomware variants, the solutions come with a number of critical limitations, which affect the applicability of the solutions in real-world scenarios. Firstly, the conversion of the PE header to image tensors, followed by the training of the CNN model, necessitates the use of dedicated GPU devices, which are very expensive for most organizations and are not available for endpoint devices (Schwartz et al., 2020). Secondly, the latency associated with the training of the CNN model is very high, which makes the detection of ransomware variants which makes real-time or near-real-time detection of ransomware variants using these CNN-based methods very difficult (Strubell et al., 2019). Finally, the image-tensor conversion pipeline, especially the CNN model, is based on the 'black box' principle, which offers very little insight into the features driving the classification process (Gibert et al., 2020).

Moreover, most of the existing literature only employs a single dataset for evaluating their models. This does not consider the issue of concept drift, which refers to the changes in the statistical properties of the target variable as ransomware is constantly developing (Jordaney et al., 2017; Pendlebury et al., 2019). This single dataset evaluation is also causing spatial overfitting in the models. Spatial overfitting refers to the ability of the model to perform well for a specific dataset but not for other datasets that have a different structure and are representative of different feature spaces. This is a significant gap in the existing literature.

It is, therefore, essential to develop an efficient, interpretable, and generalizable ransomware detection scheme that addresses the following critical needs: (a) the dependency on the GPU should be removed by using structured tabular data, (b) the performance of the scheme should be on par with or better than the performance of the deep learning models, (c) the scheme should be tested on multiple data sets to validate the robustness of the scheme with respect to the concept of data drift, and (d) the latency of the scheme should be measured and reported to validate the viability of the scheme in the real world.

1.3 Aim of the Study

The primary aim of this study will be to develop and evaluate a computationally efficient static ransomware detection framework using ensemble machine learning classifiers on tabular PE header data, thereby providing a Green AI alternative to computationally expensive deep learning approaches while maintaining or exceeding their classification accuracy.

1.4 Objectives of the Study

The following specific objectives will be pursued to achieve the aim of this study:

1. Compile and preprocess four structurally independent PE header datasets, including raw data, alternative frequency repositories, artificially engineered structural heuristics, and Import Address Table feature matrix datasets, for rigorous cross-dataset validation across datasets.

2. Implement and benchmark the performance of ten machine learning algorithms across four mathematical paradigms, including Gradient Boosting, Bagging Ensemble, Base Non-Linear, Linear/Probabilistic, for ransomware-benign classification on binary data.

3. Compare the performance of all machine learning algorithms using an exhaustive range of evaluation metrics, including accuracy, precision, recall, F1-Score, MCC, Cohen's Kappa, specificity, AUC-ROC, FPR, FNR, and training latency.

4. Compare the performance consistency across the four datasets for the machine learning algorithms, ensuring the cross-validation and concept drift robustness of the proposed approach.

5. Compare the performance efficiency and accuracy of the proposed ensemble approach with the performance of deep learning approaches, as presented in the literature, to demonstrate the Green AI advantage.

1.5 Justification for the Study

The rationale for this study is grounded in three converging imperatives. First, the rising prevalence and financial cost of ransomware attacks necessitates the development of

detection approaches that are not only accurate but also feasible in the context of the wide variety of hardware configurations that are prevalent in the real world (Beaman et al., 2021; Kapoor et al., 2022). The SOTA's reliance on GPU-accelerated deep learning architectures represents a major barrier to the adoption of the approach in organisations that lack access to this technology.

The second imperative concerns the recently emerging "Green AI" paradigm, which has highlighted the unsustainability of the current escalation of machine learning approaches (Schwartz et al., 2020). Strubell et al. (2019) have demonstrated the environmental and financial unsustainability of the current escalation of machine learning approaches, and this study will directly address this imperative by calculating the cost-benefit ratio of the ensemble machine learning approach in comparison to the CNN-based approach.

The third imperative concerns the methodological weakness of the current literature in the evaluation of ransomware detection models on a single dataset without cross-dataset validation (Moreira et al., 2023; Manavi & Hamzeh, 2022). This approach fails to account for the phenomenon of concept drift in the context of the evolving ransomware code and PE header manipulation (Jordaney et al., 2017). By using all models on the four structurally independent datasets, this study will offer a more rigorous and realistic evaluation of the models than is currently available in the literature.

1.6 Scope of the Study

The scope of this study will be limited to static analysis of Windows PE executable files for binary classification (ransomware vs. benign). The main areas that this study will focus on are:

i. Compilation and preprocessing of four independent PE header data sets from publicly available sources.

ii. Implementation and hyperparameter optimization of ten different machine learning models across four different math domains.

iii. Evaluation of classification performance using a broad set of eleven different evaluation metrics.

iv. Evaluation of training latency using a standard endpoint workstation.

v. Comparison against deep learning models using publicly available literature.

The scope of this study will not include dynamic analysis (sandbox-based behavioural analysis), ransomware family classification (multi-class classification), mobile or non-Windows malware detection. The scope is deliberately confined to the static PE header analysis domain in order to demonstrate the computational efficiency thesis.

1.7 Methodology

The proposed approach for the present research will rely on experimental and quantitative benchmarks for the evaluation of the classification and efficiency of various machine learning algorithms for the detection of ransomware. The proposed methodology for the present research will include the following steps:

1. The four publicly available data sets for PE header will be collected, preprocessed, and cleaned using the Pandas Library, where the non-numeric identifiers such as MD5 and SHA-256 will be removed, and the missing values will also be taken care of.

2. The label encoding will be performed using the LabelEncoder for the encoding of the target labels, and the standardization will also be performed using the StandardScaler for the reduction of the high magnitude variance of the structural features.

3. The data sets will be split into training and testing sets with the application of the strict train-test split with an 80-20 ratio.

4. Ten machine learning models will be applied for the classification, such as XGBoost, LightGBM, CatBoost, Random Forest, Extra Trees, Decision Tree, SVM, KNN, Logistic Regression, and Gaussian Naive Bayes.

5. The performance of the machine learning models will be evaluated based on the accuracy, precision, recall, F1 measure, Matthews correlation coefficient, Cohen's kappa statistic, specificity, Area under the ROC Curve, FPR, FNR, and training time.

6. The performance will also be compared with the proposed approach using deep learning models such as Xception CNN and LSTM proposed by Moreira et al. (2023) and other related works.

7. The experiments will be performed on a workstation such as Dell Latitude E7270, which doesn't have access to cloud-based GPU services, thus proving the proposed hypothesis of Green AI.

1.8 Definition of Terms

1. Ransomware: A type of malicious software that uses encryption to deny access to victim files unless a ransom is paid in exchange for a decryption key (Beaman et al., 2021).

2. Portable Executable (PE): The default format used by the Windows OS to store executable files, object code, and Dynamic-Link Libraries (DLLs). The PE header holds information related to the file structure and characteristics (Moreira et al., 2023).

3. Static Analysis: A malware analysis method that focuses on analyzing the structural characteristics of a binary without executing it. This is used to quickly identify malware (Ucci et al., 2019).

4. Ensemble Learning: A machine learning approach in which multiple classifiers called base learners are combined to achieve an improved classifier that generalizes better, reduces variance, and improves predictive accuracy (Breiman, 2001).

5. Gradient Boosting: An ensemble learning method in which additional classifiers are trained sequentially to correct errors in the previous classifiers using gradient descent optimization (Chen & Guestrin, 2016).

6. Green AI: A new paradigm for conducting AI-related research that emphasizes and advocates for the development of more efficient, accessible, and sustainable AI systems over the uncontrolled scaling of computational resources (Schwartz et al., 2020).

7. Concept Drift: A situation in which the statistical properties of the target variable being predicted by a model change over time, thereby causing a deterioration in the model's performance (Jordaney et al., 2017).

8. Feature Engineering: The practice of using domain expertise to generate new features from raw data that enhance the discriminative ability of machine learning algorithms.

9. Shannon Entropy: A measure of randomness or uncertainty in a given distribution, often used in malware analysis to identify packed, encrypted, or obfuscated code sections (Vidyarthi et al., 2019).

10. Import Address Table (IAT): A data structure within a PE file that holds the memory addresses for functions called from external libraries, thereby serving as indicators of API usage and malicious activities (Schultz et al., 2001).

1.9 Significance of the Study

This research will make several significant contributions to the fields of cybersecurity and applied machine learning. Firstly, it will offer empirical evidence that computationally lightweight ensemble models may achieve comparable or even better ransomware detection accuracy compared to deep learning architectures, thus proving the viability of the 'Green AI' concept within the ransomware detection problem domain. Such a finding will have direct practical implications for cybersecurity professionals, ransomware detection software development, and any organization seeking to effectively detect ransomware attacks on standard hardware infrastructure without the need for costly GPU investment. Secondly, the proposed cross-dataset evaluation approach will set a new standard for ransomware detection research, highlighting the significance of concept drift resistance for ransomware detection algorithms and encouraging future researchers to test and evaluate their approaches on multiple datasets. Finally, the proposed evaluation of all ten algorithms across four mathematical paradigms will offer a taxonomy for algorithm selection in PE-based malware detection, helping practitioners select the most appropriate model for their specific deployment needs. The evaluation of the proposed approach will also include an analysis of the computational training latency, an important but often overlooked evaluation criterion.

CHAPTER TWO

LITERATURE REVIEW

The detection of ransomware via computerized methods has been a topic of interest for more than a decade now, primarily due to the rising number and monetary losses caused by ransomware attacks on organizations and individuals across the globe. In this chapter, a comprehensive overview of relevant literature will be provided, arranged thematically to establish the concepts, trace the development and progression of detection methods, and finally pinpoint the gap in literature and position this study within the broader context. The ensuing sections will detail the evolution from dynamic to static analysis, critically review the computational limitations of current deep learning-based approaches, and substantiate the need for a "Green AI," ensemble-based detection framework.

2.1 Conceptual Framework of Machine Learning-Based Ransomware Detection

This section outlines the foundational concepts that underpin the proposed ransomware detection framework, including the technical hierarchy of concepts ranging from malware analysis paradigms to specific algorithmic families, and situates these concepts within the requirements of endpoint security.

Ransomware

Ransomware is a type of malware that restricts victims' access to their data or systems until a ransom is paid. The ransom is usually paid using cryptocurrency to ensure the attacker's anonymity. The most common type of ransomware, crypto-ransomware, uses robust symmetric or asymmetric encryption techniques to encrypt victims' data, making them unrecoverable without the attacker's private decryption key (Beaman et al., 2021). Unlike other types of malware, which aim to compromise system functionality, exfiltrate data, or establish backdoors, ransomware targets the intangible value of information, which is arguably the most important information asset for individuals and organisations (Moreira et al., 2023). The attack lifecycle of a ransomware attack includes delivery, installation, privilege escalation, file enumeration and encryption, displaying the ransom note, and optional data exfiltration for double extortion (Oz et al., 2022). Ransomware-as-a-Service (RaaS) has become a significant threat actor, allowing non-technical cybercriminals to deploy

sophisticated ransomware variants using subscription-based services on the dark web (Meland et al., 2020).

Malware Analysis Paradigms

Malware detection, including ransomware, is performed computationally and falls into three different malware analysis paradigms: static analysis, dynamic analysis, and hybrid analysis (Ucci et al., 2019). In static analysis, the structural characteristics of a binary file are examined without executing it. The structural characteristics examined include PE header information, opcode frequency, byte-level n-gram analysis, string signatures, control flow graphs, and import and export information (Moreira et al., 2023). In dynamic analysis, a binary file is executed in a sandbox environment and monitored for various characteristics, including system call information, file system access, registry access, and network communication (Fernando et al., 2020). Hybrid analysis is a combination of static and dynamic analysis, which helps to avoid the limitations and advantages of individual analysis techniques (Oz et al., 2022). However, this study will only focus on static analysis, which provides various advantages in terms of speed and safety. Figure 2.1 illustrates the taxonomy of these three analysis paradigms and their associated feature types.

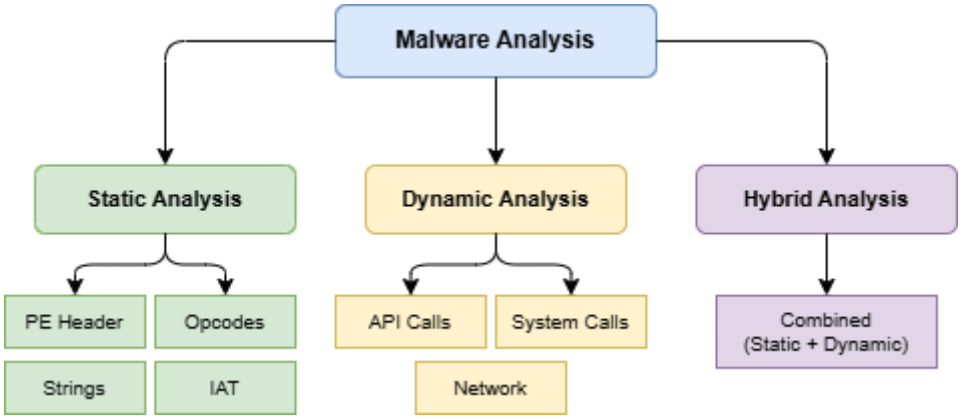


Figure 2.1: Taxonomy of Malware Analysis Approaches for Ransomware Detection

The Portable Executable (PE) File Format

The Portable Executable format is the native executable binary format for the Microsoft Windows operating system, including executables, Dynamic-Link Libraries, and other binary types. The PE header is the structured container for the metadata, including the MS-DOS Header, MS-DOS Stub, PE Signature, COFF File Header, Optional Header with its Standard Fields, Windows-Specific Fields, and Data Directories, and Section Headers (Moreira et al.,

2023). The information in the PE header fields is critical for the operation of the binary, including its architecture, entry point, code size, data size, memory layout, imported libraries, and section information. Research has established the presence of ransomware patterns in the PE header fields, including abnormal entropy values, abnormal section names, abnormal imported DLLs for cryptographic activities, and abnormal compilation timestamps, which differentiate ransomware from clean software (Vidyarthi et al., 2019; Poudyal et al., 2019).

Machine Learning for Malware Detection

Machine Learning (ML) is the computational theory that supports automated pattern recognition in PE header data, where ML classifiers can be trained on labelled examples and apply this knowledge to new examples (Ucci et al., 2019). Classical ML techniques such as Random Forest, Support Vector Machines, Logistic Regression, and Naïve Bayes have been popular for performing binary malware classification based on engineered feature vectors (Schultz et al., 2001; Kolter & Maloof, 2006; Shafiq et al., 2009). The major advantage of classical ML is its computational efficiency and interpretability on structured tabular data without any need for complex pre-processing, unlike deep learning (Gibert et al., 2020). However, classical ML is heavily reliant on engineered features that are often based on domain knowledge and require human experts to design relevant features that describe malicious behavior.

Ensemble Learning

Ensemble learning is another ML technique that is based on the concept of learning multiple models and combining them to reduce variance and/or bias in ML models and improve their performance (Breiman, 2001). Ensemble learning can be performed through bagging and boosting. Bagging is based on majority voting of multiple models trained on bootstrap samples of the training data (Breiman, 2001; Geurts et al., 2006), whereas boosting is based on sequentially training multiple models on different subsets of training data to reduce errors (Chen & Guestrin, 2016; Ke et al., 2017; Prokhorenkova et al., 2018). Recent developments in ensemble learning techniques such as XGBoost (Chen & Guestrin, 2016), LightGBM (Ke et al., 2017), and CatBoost (Prokhorenkova et al., 2018) have shown promising results on tabular data benchmarks and often surpass deep learning techniques.

Green AI and Computational Efficiency

The concept of "Green AI" was created due to the growing concern about the increasing costs of computation in AI research (Schwartz et al., 2020). The authors of the paper suggested that there should be a differentiation between "Red AI" as a way to increase the accuracy of the results by increasing the computation (more data, more models, more training), and "Green AI" as a way to increase the efficiency of the computation to achieve the highest accuracy for the lowest number of floating point operations. The impact of the computation involved in the training of the models was quantified by Strubell et al. (2019), who showed that a single execution of NLP models can result in more than 284 tonnes of CO2 emissions. In the case of ransomware detection, the concept of Green AI implies that instead of relying on GPU-based CNN models for the purpose of ransomware detection, there should be a shift towards the adoption of lightweight models that can produce similar results using common hardware.

2.2 Static Analysis and PE Header-Based Detection Methods

The analysis of the header information of the PE file for the detection of malware and ransomware has a strong body of prior research. Schultz et al. (2001) provided the foundational study in this context by demonstrating the effectiveness of the application of data mining techniques on the features of the PE file, like DLL imports, function calls, and byte sequences, in the context of the detection of never-before-seen malware. Kolter and Maloof (2006) extended this study by using multiple ML models like boosted decision trees and SVMs on the byte n-gram features of the executable files and demonstrated the effectiveness of the approach with detection accuracy of over 97%.

Shafiq et al. (2009) proposed the concept of PE-Miner, which is an automated framework for the detection of malicious PE file executables using structural information like section attributes, import/export tables, and resource directory of the PE header information. They demonstrated the effectiveness of the application of multiple ML models and the detection accuracy of the approach with low computational overhead using a few carefully engineered PE file features. This study provided the important precedent for the current study in the context of the application of tabular features of the PE file for the binary classification problem.

In the context of ransomware detection, Poudyal et al. (2019) used PEFile analysis on ransomware samples, extracting header information such as the packer used, compile time,

use of cryptographic functions, and DLL dependencies. The analysis proved the hypothesis that static PE header information is an important discriminative feature for identifying ransomware characteristics. However, the research was limited to only one ransomware family, i.e., Locky. Vidyarthi et al. (2019) extended the scope of the research by identifying 60 static PE features, such as file entropy, section information, registry modification commands, and network communication DLLs, which differentiate ransomware from benign executables and other types of malware. The researchers used the Random Forest, Decision Tree, and Naive Bayes classifiers, where the best performance was recorded for the Random Forest classifier, along with contemporary anti-malware detection approaches. However, the approach demands considerable domain knowledge for extracting the 60 features from the PE header.

2.3 Deep Learning Approaches for Ransomware Detection via PE Visualisation

The interest in applying deep learning to PE-based malware detection has been fueled by the observation that different malware families show visually distinct patterns when their PE files are mapped to images (Nataraj et al., 2011). Nataraj et al. (2011) initiated the concept of malware visualization by transforming PE files to grayscale images and observed that PE files belonging to the same family showed similar images. This laid the groundwork for applying computer vision techniques for malware detection.

Manavi and Hamzeh (2021) proposed an LSTM neural network to classify PE headers for ransomware detection and tested their system on 1,000 ransomware and 1,000 clean PE files. The best-performing model showed 93.25% accuracy, precision of 93.33%, recall of 93.25%, and an F-measure of 93.24%. The authors proposed another system based on graph embedding that used PE headers transformed to an eigenspace through Power Iteration and compared its performance with that of a Random Forest classifier (Manavi & Hamzeh, 2022a). The third contribution of the authors used CNNs to classify zigzag patterned images of PE files and showed 93.33% accuracy for a 12-layer CNN (Manavi & Hamzeh, 2022).

Moreira et al. (2023) have extended this line of research by presenting the Xception ColSeq approach, which converts the first 1,024 bytes of PE header data into colour images using a sequential vector pattern and a reverse colour palette, and then uses the Xception convolutional neural network architecture to classify these images. On their curated Dataset 2, consisting of 1,023 ransomware and 1,134 benign applications, the authors have shown

that this approach achieves an accuracy of 98.20%, precision of 97.50%, recall of 98.76%, and F-measure of 98.12%. Furthermore, the authors have also shown that six ML algorithms can be used to classify the tabular data with high accuracy. Specifically, the authors have used the six ML algorithms to classify the tabular data and have shown that the Random Forest algorithm achieves 96.51% accuracy, and the SVM algorithm achieves 96.48% accuracy. This result shows the potential of tabular ML approaches for malware classification, which is a crucial motivation for this study.

The authors Saxe and Berlin (2015) have proposed a malware classification system using a deep neural network architecture based on two-dimensional binary visualization. In this study, the authors have shown the effectiveness of using neural network architectures for PE-based visualization features. Raff et al. (2018) have proposed a neural network architecture called MalConv, which uses raw byte-level features of entire executable files. This approach achieves high detection rates but requires significant computational resources.

The above CNN-based approaches have three limitations, which this study intends to resolve, namely, the reliance of the approaches on GPU devices for training and inference, the long latency of the approaches, which is incompatible with real-time endpoint detection, and the lack of transparency of the decision-making process, which makes it difficult to forensically analyze the results of the classification (Gibert et al., 2020).

2.4 Traditional and Ensemble Machine Learning Approaches

The performance of the conventional ML algorithms has already been tested for malware and ransomware detection, proving to be competitive while requiring much less computational resources than the DL-based approaches. Bae et al. (2020) presented a systematic evaluation of the performance of six different ML algorithms, namely Naïve Bayes, KNN, Logistic Regression, Random Forest, Stochastic Gradient Descent, and SVM, for ransomware detection using behavioural features. The results proved that the Random Forest and SVM algorithms provide the highest accuracy for ransomware detection, while the Random Forest algorithm offers strong immunity to overfitting, which is a common problem in security-related datasets due to the presence of noise.

Anderson and Roth (2018) proposed the EMBER (Endgame Malware BEnchmark for Research) dataset, which contains more than a million feature vectors for PE file-related

information along with the corresponding labels, and tested the performance of the LightGBM classifier. The results proved that the gradient boosting-based approach can achieve detection accuracy of more than 98%, while the training time required is orders of magnitude less than the CNN-based approaches, which can be a strong baseline for PE-based malware detection approaches.

The gradient boosting-based approaches have proved to be very effective for cybersecurity-related applications. The XGBoost (Chen & Guestrin, 2016) algorithm introduced the regularised gradient boosting method, which is optimised for system-level performance, making it one of the most popular machine learning algorithms used for a wide range of applications, including cybersecurity. The LightGBM (Ke et al., 2017) algorithm improved the training speed of the XGBoost algorithm by using a histogram-based method for computing the gradients, which results in comparable accuracy while requiring a much shorter training time—a property of direct relevance to the Green AI thesis of this research paper. The CatBoost (Prokhorenkova et al., 2018) algorithm introduced the ordered boosting method, which can directly handle categorical features, reducing the feature engineering requirements while maintaining competitive performance with minimal hyperparameters required to be tuned.

Sgandurra et al. (2016) suggested a machine learning method for ransomware classification based on Regularised Logistic Regression and behavioural features extracted during dynamic analysis. The authors created and publicly released a dataset comprising 582 ransomware samples from 11 different families and 942 benign applications with 30,967 features. The proposed method for testing and detecting new ransomware families is based on separating one family for testing and training on the rest. The proposed method showed a 93.30% average for weighted accuracy, although the testing methodology was limited by evaluating exclusively with positive samples.

Zahoora et al. (2022) suggested a Zero-Shot Learning (ZSL) method for detecting new ransomware families. The proposed method is based on a Deep Contractive Autoencoder-based attribute learning technique combined with a heterogeneous voting ensemble, called DCAE-ZSL-HVE. The proposed method is better than Sgandurra et al.'s (2016) method for detecting new ransomware families, even though they tested only four out of eleven available families and did not maximise the number of families for training.

2.5 Concept Drift and Temporal Robustness in Malware Classification

Concept drift, or the way in which the statistical properties of the target class change over time, is one of the fundamental issues to be addressed in malware detection systems (Jordaney et al., 2017). With the way in which ransomware developers continue to evolve and change their code to avoid detection, features that were once discriminative may no longer be relevant or may even be misleading. Jordaney et al. (2017) proposed a framework for detecting concept drift in malware classification classifiers using conformal evaluators and showed that classifiers trained on historical data exhibit significant performance degradation when tested on temporally disjoint test sets.

Pendlebury et al. (2019) formalized this problem and proposed TESSERACT, a methodology for removing experimental bias in malware classification tasks by ensuring temporal consistency in training and test sets. Their results showed that many results reported in malware detection tasks were biased due to spatial bias, or the unintentional inclusion of temporally overlapping samples in training and test sets, and that temporal consistency significantly reduced reported accuracy levels.

In this paper, the problem of concept drift will be faced in a different but complementary manner to the ones presented in the previous works. While in the other works the dataset is divided into a training and a test set according to temporal consistency, all the algorithms will be tested on four different and independent datasets, each of which will represent a different feature space and a different method of data collection. This will be a strong test of temporal robustness and whether discriminative patterns transfer across fundamentally different representations of PE header information.

2.6 Computational Efficiency and the Green AI Imperative in Cybersecurity

The computational cost of machine learning models has been identified as a key consideration within the broader artificial intelligence community and within the cybersecurity domain (Schwartz et al., 2020). The Green AI movement is based upon the idea that "the pursuit of new efficiency metrics—accuracy per floating point operation, per watt, or per dollar—should be equally important to traditional metrics such as accuracy." This is especially important for ransomware detection, which needs to run on resource-constrained endpoint environments and has strict latency requirements. Moreira et al. (2023) showed that

their Xception ColSeq CNN model needed to train for an average time of 123.38s per cross-validation fold on a Google Colab environment with Tensor Processing Units (TPUs), with a minimum and maximum number of training epochs being 14 and 20, respectively. However, the training times for all the ML algorithms they tested, including Random Forest (9.00s), SVM (7.12s), KNN (4.73s), LR (2.22s), and Naive Bayes (0.67s), were one to two orders of magnitude faster. Crucially, however, the difference in accuracy between the best-performing ML algorithm, Random Forest at 96.51%, and the CNN, Xception ColSeq at 98.20%, is only 1.69%, and one is left to wonder if this is worth the significantly greater computational cost. Anderson and Roth (2018) showed that LightGBM, trained on the EMBER dataset, could deliver malware detection results similar to commercial anti-virus engines but only needed to perform CPU calculations, making it appropriate for deployment on resource-constrained environments. This observation aligns with broader findings in the machine learning community that gradient boosting methods consistently outperform deep learning on structured tabular data across multiple independent benchmarking studies (Grinsztajn et al., 2022).

2.7 Challenges and Research Gaps

Yet, in spite of the significant advancements in ransomware detection research in recent years, some important challenges and gaps have been identified:

Single-Dataset Evaluation Bias: The vast majority of studies have only tested their approach using a single dataset. This means that the generalizability and resistance to concept drift of the approach cannot be guaranteed (Moreira et al., 2023; Manavi & Hamzeh, 2022; Bae et al., 2020).

Computational Cost Neglect: Very few studies have focused on the computational cost of their approach. This is surprising, given its importance in real-world ransomware detection scenarios (Schwartz et al., 2020).

Over-Reliance on Deep Learning: The current trend in ransomware detection research is to develop more and more complex deep learning architectures. However, this trend might not be in the best interest of organizations that need to develop ransomware detection approaches that can be deployed in real-world scenarios (Gibert et al., 2020). In fact, the assumption that more complex models will produce better results for tabular PE-based ransomware detection has not been rigorously tested.

Limited Algorithm Diversity: Most comparative studies have only compared three to six machine learning algorithms. This means that the entire spectrum of machine learning algorithms for PE-based ransomware detection has not been thoroughly explored (Bae et al., 2020; Moreira et al., 2023).

Interpretability Deficit: CNN-based approaches have very poor interpretability in terms of which PE header fields are used for making the final prediction (Gibert et al., 2020).

2.8 Summary of Reviewed Literature

The literature review has identified that the ransomware detection field using ML techniques has followed a well-defined evolutionary path, ranging from traditional rule-based and signature-based approaches to classical ML techniques using manually derived features, and ultimately to the dominance of deep learning-based techniques using visualised PE data. Although the latter, especially PE header image classification using CNNs, has shown the highest accuracy rates, these are accompanied by high computational demands and offer no interpretability. Classical ML techniques using gradient boosting models have shown high accuracy rates with significantly lower computational demands but have not been thoroughly evaluated using multiple data sets and compared against all available algorithm options.

The proposed study will fill these identified knowledge gaps by: (1) conducting the most comprehensive algorithm benchmarking study for PE-based ransomware detection to date using ten algorithms across four different mathematical paradigms; (2) introducing cross-dataset validation using four structurally independent data sets to evaluate the robustness against concept drift; (3) measuring and reporting computational latency to validate the Green AI hypothesis; and (4) showing that classical ML models can achieve equal or even better results than deep learning models using tabular PE data without the need for any GPU hardware.

Table 1: Summary of Reviewed Literature

S/N	Author(s)	Year	Contribution	Methodology	Gap	Limitation
1	Moreira, Moreira & de Sales Jr.	2023	Improved ransomware detection using	Xception CNN on sequential colour images	No cross-dataset validation	Requires GPU/TPU; image

			Xception CNN on PE header colour images	from PE header bytes		conversion overhead
2	Manavi & Hamzeh	2021	Static ransomware detection using LSTM on PE header byte sequences	LSTM network on raw 1024-byte PE header	Did not test new family detection	Limited to single dataset; 93.25% accuracy
3	Manavi & Hamzeh	2022	PE header-based ransomware detection using CNN with grey-scale zigzag images	Custom 12-layer CNN on 32×32 grey-scale images	Suboptimal colour palette and vector pattern	Did not test new family detection
4	Sgandurra et al.	2016	EldeRan: ML-based ransomware classification using behavioural features	Regularised Logistic Regression on dynamic features	Incomplete testing methodology for new families	Test set contained only positive samples
5	Zahoora et al.	2022	Zero-day ransomware detection using deep contractive autoencoder and ensemble	DCAE-ZSL with heterogeneous voting ensemble	Only tested 4 of 11 families	Did not maximise training families
6	Bae et al.	2020	Ransomware detection using six ML algorithms on	NB, KNN, LR, RF, SGD, SVM comparison	Limited algorithm diversity	Single dataset evaluation

			behavioural features			
7	Vidyarthi et al.	2019	Identified 60 static PE properties distinguishing ransomware	RF, Decision Tree, Naïve Bayes on engineered PE features	Requires expert feature extraction	Needs significant PE header knowledge
8	Poudyal et al.	2019	PEFile analysis technique for ransomware study	Static analysis of PE header sections	Only one family analysed	Limited to Locky family
9	Anderson & Roth	2018	EMBER: Open dataset and LightGBM baseline for PE malware detection	LightGBM on 2,381 PE features	General malware, not ransomware-specific	Limited to single gradient boosting model
10	Gibert et al.	2020	Survey on ML/DL for malware detection and classification	Comprehensive literature review	Review-only ; no experimental contribution	No new empirical results
11	Zhang et al.	2019	Ransomware family classification using N-gram opcodes and ML	RF, KNN, SVM on opcode N-gram features	Focuses on family classificatio n, not detection	Requires disassembly for opcode extraction
12	Nataraj et al.	2011	Malware visualisation and automatic classification via image similarity	KNN on GIST features from binary images	Early work; limited classifier diversity	Grey-scale only; limited to visual similarity

13	Saxe & Berlin	2015	DNN-based malware detection using 2D binary visualisation	Deep neural network on PE features	No ransomware-specific evaluation	High computational requirements
14	Schultz et al.	2001	Foundational data mining methods for malware detection using PE features	RIPPER, NB, Multi-NB on PE headers and DLL imports	Dated; limited to early malware types	Small dataset; limited algorithm scope
15	Kolter & Maloof	2006	Learning to detect malicious executables using byte N-grams	Boosted DT, SVM, NB on N-gram features	Five-year dataset may exhibit drift	Computationally expensive N-gram extraction
16	Shafiq et al.	2009	PE-Miner: Automated malicious PE detection using structural features	Multiple ML classifiers on PE structural features	General malware; not ransomware-focused	Limited feature set from PE structure
17	Raff et al.	2018	MalConv: Neural network processing raw executable bytes	1D CNN on raw byte sequences of entire executables	Extremely high computational cost	Requires GPU; processes entire binary
18	Al-rimy et al.	2018	Comprehensive survey on ransomware threat factors and	Systematic literature review and taxonomy	Review-only; no experimental results	Does not evaluate specific algorithms

			countermeasures			
19	Jordaney et al.	2017	TRANSCEND: Framework for detecting concept drift in malware classification	Conformal evaluators for drift detection	Focused on drift detection, not classification	Does not propose improved classifiers
20	Pendlebury et al.	2019	TESSERACT: Eliminating temporal bias in malware classification evaluation	Temporal partitioning methodology	Methodological contribution; no new models	Does not address cross-dataset validation
21	Kok et al.	2022	Early detection of crypto-ransomware using pre-encryption algorithm	Pre-encryption detection using ML	Focused on behavioural indicators	Requires partial execution monitoring
22	Fernando et al.	2020	Survey on evolution of ransomware detection using ML and DL	Comprehensive review of ML/DL techniques	Review-only; no empirical contribution	Notes that datasets are rarely shared
23	Oz et al.	2022	Survey on ransomware evolution, taxonomy, and defence solutions	Systematic survey with defence taxonomy	Review-only; comprehensive but non-empirical	Does not propose new detection method
24	Beaman et al.	2021	Ransomware: Advances, analysis, challenges,	Survey with analysis of recent ransomware trends	Identifies challenges but no algorithmic solution	Recommendations are general

			and future directions			
25	Ucci et al.	2019	Survey of ML techniques for malware analysis	Comprehensive review of feature types and algorithms	General malware scope; no ransomware focus	Does not evaluate Green AI considerations
26	Moussaileb et al.	2021	Survey on Windows-based ransomware taxonomy and detection mechanisms	Systematic review of detection mechanisms	Review-only; identifies gaps but no solution	Limited to Windows platform analysis
27	Kapoor et al.	2022	Ransomware detection, avoidance, and mitigation: Review and future directions	Review with mitigation strategy taxonomy	No experimental validation	General recommendations
28	Meland et al.	2020	Analysis of the Ransomware-as-a-Service economy within the darknet	Qualitative analysis of RaaS ecosystem	Threat analysis; no detection methodology	Does not propose countermeasures
29	Schwartz et al.	2020	Green AI: Advocating computational efficiency in AI research	Position paper proposing efficiency metrics	Conceptual framework; no empirical validation	Does not apply to cybersecurity specifically
30	Strubell et al.	2019	Energy and policy considerations for deep	Quantified carbon emissions of DL training	Focused on NLP; not cybersecurity	Does not benchmark lightweight alternatives

			learning in NLP			

2.9 The PE Header as a Feature Source for Ransomware Detection

The PE header's value in the feature extraction process for ransomware detection lies in the fact that it serves as the blueprint of the executable file in the Windows operating system. The header contains metadata that is critical in the sense that it represents the design decisions of both the software developer and the ransomware developer (Moreira et al., 2023). There are two different paradigms in the use of the information in the PE header: the raw byte paradigm and the parsed field paradigm.

In the raw byte paradigm, the first **N** bytes of the PE header are considered as numerical data without any parsing of the header fields. This paradigm has the advantages of not requiring any knowledge of the PE header format and not requiring the differentiation between variable length fields. This paradigm has also been used in the works of Moreira et al. (2023) and Manavi and Hamzeh (2021, 2022). However, this paradigm loses the semantic information in the header fields.

In the parsed field paradigm, the different fields of the PE header, like the SizeOfCode, SizeOfInitializedData, AddressOfEntryPoint, NumberOfSections, SectionAlignment, and FileAlignment, are extracted and presented in a tabular form with specific names. The advantages of the parsed field paradigm include the maintenance of the semantic meaning of the header fields, as well as the applicability of feature engineering, like the calculation of the Shannon entropy of the different sections of the executable file. The evaluation of the two paradigms will be done by the proposed study, considering its multi-dataset evaluation, where Datasets 1 and 2 are for the raw byte paradigm, while Datasets 3 and 4 are for the parsed field paradigm.

2.10 Limitations of Deep Learning for Endpoint Ransomware Detection

Although deep learning approaches yield the highest accuracy values in PE-based ransomware detection approaches, there exist some practical limitations that render such approaches less viable in terms of practical application to standard endpoint devices.

Firstly, there exists an additional preprocessing step in converting raw byte vectors into two-dimensional images in colour or greyscale format. This additional step is unnecessary as it does not provide any additional discriminative information. The raw bytes provide all necessary information; it is simply re-organised in image format to allow CNN processing.

Secondly, training CNNs requires forward and backward propagation of dozens of layers of convolutional and pooling layers. This is computationally intensive and requires significant memory resources, often requiring GPU acceleration to allow training within reasonable time frames (Gibert et al., 2020).

Thirdly, due to the black box nature of deep learning approaches, it is difficult to ascertain which particular PE header fields or bytes influence classification decisions (Schwartz et al., 2020). While approaches such as LayerCAM, used by Moreira et al. (2023), provide an indication of image regions of interest in image-based approaches, such approaches remain inherently approximate.

CHAPTER THREE

RESEARCH METHODOLOGY

3.1 Introduction

In this chapter, the methodology to be adopted for developing and evaluating the proposed computationally efficient framework for detecting ransomware attacks will be discussed in detail. The proposed methodology is divided into five different phases, including dataset compilation and preprocessing, model implementation and training, performance evaluation, cross-dataset validation, and computational efficiency analysis. Each of these phases will be discussed in sufficient detail to enable complete replicability of the proposed methodology.

The following sections will discuss the following topics in detail:

Research Design: The proposed experimental methodology and research philosophy to be adopted for conducting this study will be discussed in this section.

Dataset Description: The proposed methodology for describing the four independent PE header datasets to be used for conducting this study will be discussed in this section. The description will include information about the proposed dataset compilation and its appropriateness for conducting this study.

Data Preprocessing Pipeline: The proposed data preprocessing pipeline to be adopted for this study will be discussed in this section.

Machine Learning Algorithms: The proposed methodology for describing the ten machine learning algorithms to be adopted for this study will be discussed in this section. Each of these algorithms will be described based on their mathematical family and will be presented along with their formulations.

Evaluation Metrics: The proposed methodology for conducting the performance and efficiency analysis will be discussed in this section.

Comparative Analysis: The proposed methodology for conducting comparative analysis will be discussed in this section.

Summary: The proposed methodology for conducting this study will be summarized in this section.

3.2 Research Design

The research design will be based on a quantitative experimental approach and will be structured as a multi-algorithm and multi-dataset benchmarking study. The research methodology will be based on a four-stage process, as shown in Figure 3.1.

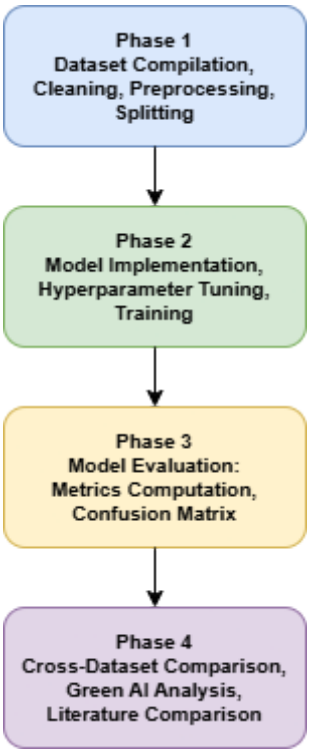


Figure 3.1: Overview of the Research Design for Computationally Efficient Ransomware Detection

Phase 1: Data Compilation and Preprocessing. Four PE header datasets will be compiled and processed. The datasets will be structurally independent. Non-numeric identifiers will be eliminated. Missing values will be processed. Target labels will be encoded. Feature values will be standardized.

Phase 2: Model Implementation and Training. Ten machine learning algorithms will be implemented using the Scikit-learn library and the XGBoost, LightGBM, and CatBoost libraries. Each algorithm will be trained using the 80% training set of each dataset. Hyperparameters will be set to default values.

Phase 3: Performance Evaluation. Each model will be evaluated using the 20% test set of each dataset. Eleven evaluation metrics and training latency will be computed for each model. The confusion matrix will be computed for each model.

Phase 4: Comparative Analysis. Results will be compared across datasets. Results will be compared to deep learning baseline results by Moreira et al. (2023), and to results by Manavi and Hamzeh (2021, 2022).

3.2.1 Proposed Ransomware Detection System Algorithm

The following pseudo code will define the entire experimental process:

Algorithm 1: Computationally Efficient Static Ransomware Detection Pipeline

Input (D): Four PE header data sets {D_1, D_2, D_3, D_4}, each with feature matrix X and binary target vector y

Output (R): Comparative performance matrix for all models and data sets

```
1.  FOR each dataset d ∈ {D₁, D₂, D₃, D₄}:
2.      Load dataset d using Pandas
3.      Drop non-numeric columns (MD5, SHA-256, filenames)
4.      Drop rows with NaN/null values
5.      Encode target variable using LabelEncoder: y → {0, 1}
6.      Standardise features using StandardScaler: X → X_scaled
7.       Split: X_train, X_test, y_train, y_test ← train_test_split(X_scaled,
y, test_size=0.20)
8.
9.      Define model set M = {XGBoost, LightGBM, CatBoost,
10.                           Random Forest, Extra Trees,
11.                           Decision Tree, SVM, KNN,
12.                           Logistic Regression, Gaussian NB}
13.
14.     FOR each model m ∈ M:
15.         Record start_time
16.         Train m on (X_train, y_train)
17.         Record end_time
18.         training_time = end_time - start_time
19.
20.         Generate predictions: y_pred = m.predict(X_test)
21.         Generate probabilities: y_prob = m.predict_proba(X_test)
22.
23.         Compute confusion matrix: CM = confusion_matrix(y_test, y_pred)
24.         Extract TP, TN, FP, FN from CM
25.
26.         Compute metrics: Accuracy, Precision, Recall, F1-Score,
27.                          MCC, Kappa, Specificity, AUC-ROC, FPR, FNR
28.         Store: results[d][m] = {metrics, training_time}
29.     END FOR
30. END FOR
31.
32. Aggregate results across datasets
33. Compare with DL baselines from literature
34. Return R: Comparative performance matrix
```

Algorithm 1: Computationally Efficient Static Ransomware Detection Pipeline

3.3 Dataset Description

Four data sets with different structural characteristics will be used, and these data sets are structurally independent of each other. This will guarantee the evaluation of the model comprehensively and the avoidance of concept drift. Table 2 will show the characteristics of the data sets used in this study.

Table 2: Summary of Datasets to be Used for the Study

Property	Dataset 1	Dataset 2	Dataset 3	Dataset 4
Name	Mendeley PE Header Baseline	Alternate PE Headers	Structural Properties	API/IAT Feature Matrix
Source	Moreira et al. (2023); Mendeley Data	Independent PE header repository (pe_headers.csv)	Engineered from PE structural fields	Custom extraction from PE import tables
Feature Type	Raw first 1,024 bytes (decimal 0–255)	Raw sequential byte frequencies	Parsed structural heuristics (SizeOfCode, SectionAlignment, Shannon Entropy, etc.)	IAT-derived features (num_libraries, num_functions)
Ransomware Samples	1,023 (25 families)	~1,000	Variable	~1,000
Benign Samples	1,134	~1,000	Variable	~1,000
Total Features	1,024	1,024	~50–100 engineered features	~200–500 (library/function counts)
Label Format	Binary (Ransomware/Goodware)	Binary (Malware/Benign)	Binary	Binary

Dataset 1 (Mendeley Baseline).

The first dataset to be used for evaluation, created by Moreira et al. (2023), and it is publicly available in Mendeley Data. It has 2,157 samples in total, consisting of 1,023 ransomware executables from 25 categorized families and 1,134 benign Windows applications. Each sample is represented by a vector of 1,024 numerical features, which correspond to decimal values ranging from 0 to 255 and correspond to the first 1,024 bytes of the PE header. The ransomware executables were obtained from the VirusShare and Hybrid-Analysis databases. The categorization of the ransomware into 25 families is based on VirusTotal vendor consensus. The benign executables come from PortableApps, Softonic, Windows 10 system files, and web-based no-install software. All executables were scanned using ESET NOD32 Antivirus.

Dataset 2 (Alternate PE Headers).

The second dataset to be used for evaluation. It is a different set of raw sequential byte frequencies from PE headers and is contained in a file titled "pe_headers.csv." The addition of this second dataset will enable us to check for spatial concept drift, i.e., if models based on one set of PE header bytes generalize to another set. The original source for this dataset is Manavi and Hamzeh (2021), and it has 1,000 ransomware and 1,000 benign executables. Each executable is represented by a vector of 1,024 features representing the first 1,024 bytes of the PE header.

Dataset 3 (Structural Properties).

This dataset will include engineered structural heuristics derived from the parsed header fields of the PE file. Unlike the raw byte-level characteristics of the Dataset 1 and Dataset 2 samples, this dataset will include semantic-level characteristics like SizeOfCode, SizeOfInitializedData, SizeOfUninitializedData, AddressOfEntryPoint, BaseOfCode, SectionAlignment, FileAlignment, NumberOfSections, TimeDateStamp, and Shannon entropy values calculated on the sections of the PE file (like .text, .data, .rsrc). This dataset will validate the discriminative power of the models on structural characteristics.

Dataset 4 (Extracted API/IAT).

This dataset will include a balanced matrix of 2,000 samples, with 1,000 ransomware and 1,000 benign samples, with the inclusion of characteristics extracted from the Import Address Table (IAT) of the PE file. Feature engineering techniques will be employed to obtain

characteristics like the number of libraries (num_libraries) and the total number of functions (num_functions) in the IAT, as well as binary-level characteristics of the inclusion of specific libraries like ADVAPI32.dll, CRYPT32.dll, and WS2_32.dll and functions in the IAT. This dataset will validate the inclusion of import characteristics and the level of discriminative power of the models on the inclusion of these characteristics.

3.4 Data Preprocessing Pipeline

The preprocessing steps are to be followed uniformly for all four datasets. The preprocessing steps are to be divided into six steps. The complete preprocessing pipeline is illustrated in Figure 3.2. Each step needs to be performed on all four datasets. The steps are as follows:

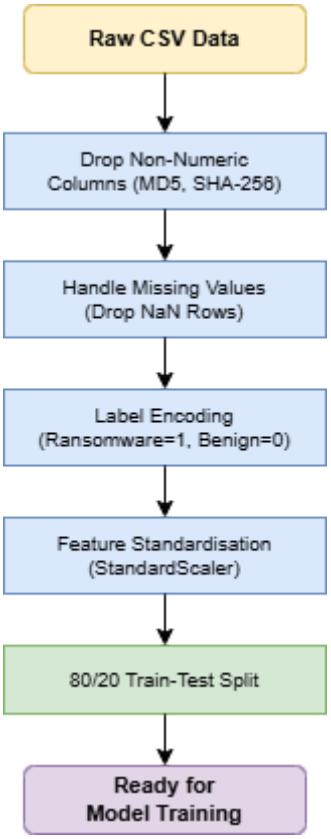


Figure 3.2: Data Preprocessing Pipeline for PE Header Datasets

Step 1: Data Ingestion. Each of the four datasets needs to be ingested into a Pandas DataFrame for further manipulation and analysis. Initial exploratory data analysis needs to be performed on each of the datasets.

Step 2: Non-Numeric Column Removal. Columns containing non-numeric information such as MD5 hashes, SHA-256 hashes, filenames, etc., need to be removed from the feature matrix. These columns do not contain any information for classification and need to be removed.

Step 3: Missing Value Handling. Rows containing NaN values in any of the feature columns need to be identified and removed. The percentage of the data being removed needs to be tracked for the completeness of the data.

Step 4: Target Variable Encoding. The target variable needs to be encoded. The target variable is currently in text format. For instance, the values Malware and Benign need to be encoded to 1 and 0, respectively. Label Encoding needs to be performed on the target variable using Scikit-learn's LabelEncoder.

Step 5: Feature Standardisation. All columns containing features need to be standardised. StandardScaler needs to be used for standardising the features so that they have zero mean and unit variance. The standardisation formula is given by:

$$X_{scaled} = \frac{X - \mu}{\sigma}$$

where μ is the mean of the feature and σ is the standard deviation of the feature. Feature standardisation is critical for gradient-based algorithms such as Logistic Regression and SVM, though it will be applied uniformly across all algorithms for consistency.

Step 6: Data Partitioning. Each of the four datasets needs to be partitioned into training and testing sets with a strict 80:20 ratio. The data partitioning strategy is depicted in Figure 3.3.

$$D = D_{train} \cup D_{test}, \quad |D_{train}| = 0.80|D|, \quad |D_{test}| = 0.20|D|$$

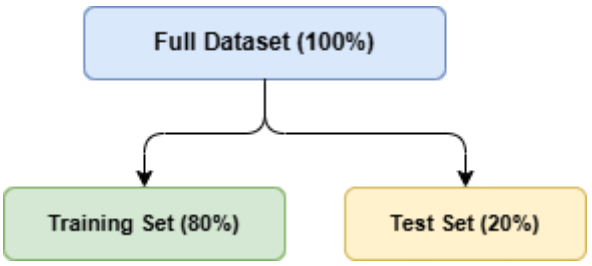


Figure 3.3: Data Splitting Strategy for Model Training and Evaluation

3.5 Machine Learning Algorithms

To ensure that all aspects of this supervised learning landscape are adequately explored, ten machine learning algorithms, divided into four mathematical families, will be implemented and tested. Figure 3.4 is a representation of this classification.

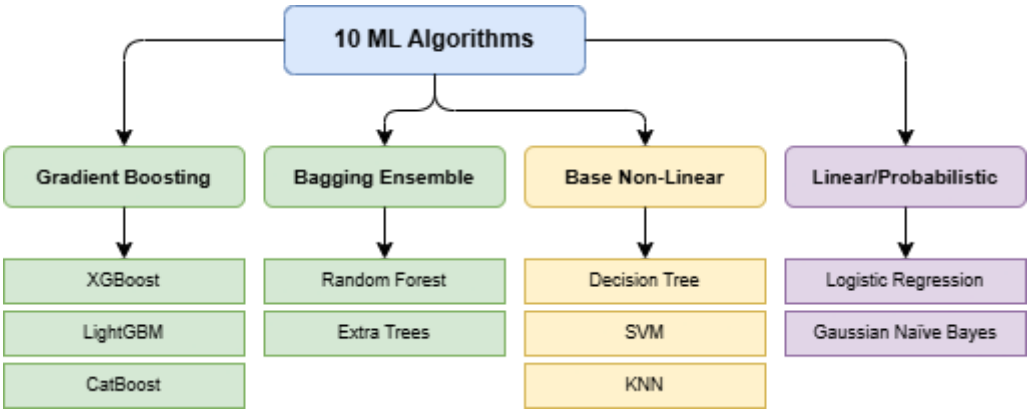


Figure 3.4: Taxonomy of the Ten Machine Learning Algorithms Across Four Mathematical Families

Table 3: Summary of Machine Learning Algorithms to be Benchmarked

Family	Algorithm	Type	Key Mechanism	Expected Strength
Gradient Boosting	XGBoost	Ensemble (Sequential)	Regularised gradient boosting with tree pruning	High accuracy; handles missing values
Gradient Boosting	LightGBM	Ensemble (Sequential)	Histogram-based leaf-wise tree growth	Fastest training; memory efficient

Gradient Boosting	CatBoost	Ensemble (Sequential)	Ordered boosting; native categorical handling	Minimal tuning; robust defaults
Bagging Ensemble	Random Forest	Ensemble (Parallel)	Bootstrap aggregation of decision trees	Robust to overfitting; handles noise
Bagging Ensemble	Extra Trees	Ensemble (Parallel)	Extremely randomised splits; no bootstrapping	Faster than RF; further variance reduction
Base Non-Linear	Decision Tree	Single Model	Recursive binary splitting on features	Fully interpretable; fast inference
Base Non-Linear	SVM	Single Model	Maximum margin hyperplane in kernel space	Effective in high-dimensional spaces
Base Non-Linear	KNN	Instance-Based	Distance-based majority voting	Non-parametric; simple implementation
Linear/Probabilistic	Logistic Regression	Linear Model	Log-odds estimation via logistic function	Interpretable; strong baseline
Linear/Probabilistic	Gaussian Naïve Bayes	Probabilistic	Bayes' theorem with Gaussian feature assumption	Extremely fast; minimal data required

3.5.1 Gradient Boosting Family

i. XGBoost (Extreme Gradient Boosting)

The XGBoost algorithm is a scalable implementation of gradient boosted decision trees that includes L1 and L2 regularisation to limit the complexity of the model and prevent overfitting. The objective function that is minimised is:

$$\mathcal{L}(\phi) = \sum_{i=1}^{n} l(y_i, \hat{y}_i) + \sum_{k=1}^{K} \Omega(f_k)$$

where l is a differentiable loss function, y is a prediction made by the model for a given sample i, K is the total number of trees in the model, and $\Omega(f_k)$ is a regularisation term for tree k. The regularisation term is defined by:

$$\Omega(f) = \gamma T + \frac{1}{2}\lambda \sum_{j=1}^{T} w_j^2$$

where T is the total number of leaves in a tree, w_j is the weight of a leaf node j, γ is a tree complexity parameter, and λ is a coefficient for L2 regularisation.

ii. LightGBM (Light Gradient Boosting Machine)

LightGBM (Ke et al., 2017) is a very efficient library for gradient boosting. LightGBM has two major innovations. The first is Gradient-Based One-Side Sampling (GOSS), where instances with high gradients are kept, and instances with low gradients are sampled randomly. The second is Exclusive Feature Bundling (EFB), where features are bundled together to reduce dimensionality. In addition, LightGBM makes use of leaf-wise best-first tree building as opposed to level-wise building in XGBoost. The former has faster convergence speed and lower memory usage.

iii. CatBoost (Categorical Boosting)

CatBoost (Prokhorenkova et al., 2018) will be used as a framework for implementing a gradient boosting model with ordered boosting, which is based on permutations and thus ensures no target leakage occurs during model training. Additionally, CatBoost supports categorical features. The PE header data is mostly numeric in nature, but CatBoost's symmetric tree model has an inbuilt resistance to overfitting through its ordered boosting method.

3.5.2 Bagging Ensemble Family

iv. Random Forest (RF)

The Random Forest algorithm, as proposed by Breiman in 2001, will be used as an ensemble classifier with bagging, where the ensemble consists of T decision trees, each trained on a

bootstrap sample of the training set with a random subset of features used at each split. The prediction made by the ensemble for a given input x is determined by majority vote, i.e.,

$$\hat{y} = \text{mode}\{h_1(x), h_2(x), \ldots, h_T(x)\}$$

where $h_t(x)$ represents the prediction made by the t-th tree in the forest for the input x. Random Forest has low bias and low variance compared to individual decision trees.

v. Extra Trees (Extremely Randomised Trees)

Extra Trees (Geurts et al., 2006) will also be used, which is similar to the random forest algorithm but adds further randomization by selecting the split point randomly for each feature rather than selecting the best one after selecting the random subset of features. This further randomization will reduce the variance but slightly increase the bias. The bootstrapping step is also skipped in Extra Trees by using the full training set for each tree, which speeds up the algorithm.

3.5.3 Base Non-Linear Family

vi. Decision Tree (DT)

One Decision Tree classifier will be included as a non-ensemble baseline, allowing the assessment of the incremental benefit of the aggregation step in the ensemble. The CART (Classification and Regression Trees) algorithm will be used, which in turn uses the Gini Impurity function:

$$Gini(t) = 1 - \sum_{k=1}^{K} p_k^2$$

where p_k denotes the proportion of class k samples at node t

vii. Support Vector Machine (SVM)

SVM will be used to determine the optimal hyperplane which has the maximum margin between ransomware and benign classes. The optimisation problem for given data (x_i, y_i) where $y_i \in \{-1, +1\}$ is given by:

$$\min_{w,b} \frac{1}{2}\|w\|^2 \quad \text{subject to} \quad y_i(w \cdot x_i + b) \geq 1, \quad \forall i$$

The Radial Basis Function (RBF) kernel will be used for non-linear class boundaries:

$$K(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|^2)$$

viii. K-Nearest Neighbours (KNN)

KNN will classify each test sample based on the majority class among its k nearest neighbors in the training set, where Euclidean or Manhattan distance can be used as the distance metric:

$$d(x, x') = \sqrt{\sum_{j=1}^{p} (x_j - x'_j)^2}$$

3.5.4 Linear/Probabilistic Family

ix. Logistic Regression (LR)

The model will use the logistic regression algorithm to estimate the probability of the ransomware class using the logistic or sigmoid function:

$$P(y = 1|x) = \frac{1}{1 + e^{-(w \cdot x + b)}}$$

The model will be trained by minimizing the binary cross-entropy loss:

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^{N} [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

x. Gaussian Naïve Bayes (GNB)

The model will use Bayes' theorem with the "naive" assumption of conditional independence between features given the class label:

$$P(y|x_1, \ldots, x_p) \propto P(y) \prod_{j=1}^{p} P(x_j|y)$$

where each class-conditional feature distribution is assumed to be Gaussian:

$$P(x_j|y) = \frac{1}{\sqrt{2\pi\sigma_{jy}^2}} \exp\left(-\frac{(x_j - \mu_{jy})^2}{2\sigma_{jy}^2}\right)$$

3.6 Evaluation Metrics and Performance Analysis

A comprehensive set of eleven metrics will be used to assess the classification performance and computational efficiency of each model. These metrics will be derived from a confusion matrix that will be calculated for every model-data combination.

Table 4: Confusion Matrix Interpretation for Binary Ransomware Classification

	Predicted Ransomware	Predicted Benign
Actual Ransomware	True Positive (TP)	False Negative (FN)
Actual Benign	False Positive (FP)	True Negative (TN)

3.6.1 Classification Performance Metrics

Accuracy calculates the overall accuracy of the model in terms of the number of correct predictions made:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Precision calculates the accuracy of the model in terms of the number of true positives among the predicted ransomware:

$$Precision = \frac{TP}{TP + FP}$$

Recall (Sensitivity) calculates the accuracy of the model in terms of the number of true positives among the actual ransomware:

$$Recall = \frac{TP}{TP + FN}$$

F1-Score calculates the harmonic mean of the precision and recall of the model:

$$F1 = \frac{2 \times Precision \times Recall}{Precision + Recall} = \frac{2TP}{2TP + FP + FN}$$

Specificity (True Negative Rate) calculates the accuracy of the model in terms of the number of true negatives:

$$Specificity = \frac{TN}{TN + FP}$$

Matthews Correlation Coefficient (MCC) calculates the accuracy of the classification model in terms of the four quadrants of the confusion matrix:

$$MCC = \frac{(TP \times TN) - (FP \times FN)}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}}$$

Cohen's Kappa (κ) calculates the accuracy of the classification model in terms of the agreement between the true and predicted labels:

$$\kappa = \frac{p_o - p_e}{1 - p_e}$$

where p_o is the observed agreement (accuracy), and p_e is the agreement observed by chance.

Area Under the ROC Curve (AUC-ROC) calculates the accuracy of the classification model in terms of the ability of the classifier to distinguish between the two classes:

$$AUC = \int_0^1 TPR(FPR)\, d(FPR)$$

False Positive Rate (FPR) calculates the accuracy of the classification model in terms of the number of false positives:

$$FPR = \frac{FP}{FP + TN}$$

False Negative Rate (FNR) calculates the accuracy of the classification model in terms of the number of false negatives—the most dangerous type of false classification in the context of malware classification:

$$FNR = \frac{FN}{FN + TP}$$

3.6.2 Computational Efficiency Metric

Training Time (seconds): This measure will be automatically calculated using Python's time module for each model. The start time will be recorded before the model is fit and the end time after the model is fit, with the difference representing the training duration. This measure will be the primary way in which the Green AI hypothesis is quantitatively tested and validated, allowing for a comparison with the training times of the CNN-based approaches in Moreira et al. (2023).

3.7 Hardware Environment and Green AI Validation

All experiments will be conducted on a standard endpoint workstation (Dell Latitude E7270) that will not have access to cloud-based GPU support, tensor processing units, or high-performance computing environments. By employing consumer-grade hardware support for this research, a deliberate research design choice is made to validate the research's underlying thesis statement that computationally efficient ensemble-based ML solutions can deliver competitive ransomware detection performance using the same class of hardware that

is commonly found in organisational environments. It is noteworthy that the absence of GPU acceleration will be highlighted in the research to underscore the deployability of this research's approach in stark contrast to GPU/TPU-supported training environments adopted by Moreira et al. (2023) and other CNN-based approaches.

3.8 Comparative Analysis with Existing Techniques

The efficacy of the proposed research methodology will be compared to existing deep learning-based approaches in the literature, and the following benchmarks will be of particular interest:

Table 5: Deep Learning Baselines for Comparative Analysis

Model	Dataset	Accuracy (%)	Precision (%)	Recall (%)	F-measure (%)	Source
Xception ColSeq (CNN)	Dataset 2 (Moreira)	98.20	97.50	98.76	98.12	Moreira et al. (2023)
Xception ColSeq (CNN)	Dataset 1 (Manavi)	93.73	92.95	94.64	93.75	Moreira et al. (2023)
LSTM	Dataset 1 (Manavi)	93.25	93.33	93.25	93.24	Manavi & Hamzeh (2021)
CNN (Grey-scale Zigzag)	Dataset 1 (Manavi)	93.33	93.40	93.33	93.34	Manavi & Hamzeh (2022)
Graph Embedding + RF	Dataset 1 (Manavi)	93.30	93.48	93.30	93.28	Manavi & Hamzeh (2022a)

The comparison will be based on three key aspects: (1) Classification performance parity with ensemble ML models to ensure accuracy at or above the confidence interval of the CNN models; (2) Computational efficiency superiority with an order-of-magnitude reduction in training times; and (3) Hardware accessibility with a demonstration of achieving competitive results without GPU/TPU infrastructure.

3.9 Summary of Methodology

This chapter has presented a rigorous and comprehensive methodology for computationally efficient ransomware detection using ensemble machine learning on tabular PE header data. The methodology covers four structurally independent datasets to validate cross-dataset generalisability, ten different machine learning algorithms belonging to four different mathematical families to ensure comprehensive algorithmic benchmarking, eleven different evaluation metrics to ensure multidimensional performance evaluation, and computational latency to validate the Green AI hypothesis. All experiments will be run on standard endpoint hardware without GPU acceleration, directly addressing the computational constraints faced in practice.

The proposed methodology will significantly contribute to the advancement of the state-of-the-art in ransomware detection by: (1) Presenting the most comprehensive algorithm benchmarking study for ransomware detection based on PE header data; (2) Introducing cross-dataset validation as a proxy to evaluate the performance in the presence of concept drift; (3) Presenting a quantitative evaluation of the computational cost-benefit ratio between ensemble ML and DL; and (4) Validating the applicability of Green AI principles in endpoint cybersecurity.

REFERENCES

Al-Rimy, B. A. S., Maarof, M. A., & Shaid, S. Z. M. (2018). Ransomware threat success factors, taxonomy, and countermeasures: A survey and research directions. *Computers & Security, 74*, 144-166. https://doi.org/10.1016/j.cose.2017.12.001

Anderson, H. S., & Roth, P. (2018). EMBER: An open dataset for training static PE malware machine learning models. *arXiv preprint arXiv:1804.04637*. https://doi.org/10.48550/arXiv.1804.04637

Bae, S. I., Lee, G. B., & Im, E. G. (2020). Ransomware detection using machine learning algorithms. *Concurrency and Computation: Practice and Experience*, 32(18), e5422. https://doi.org/10.1002/cpe.5422

Beaman, C., Barkworth, A., Akande, T. D., Hakak, S., & Khan, M. K. (2021). Ransomware: Recent advances, analysis, challenges and future research directions. *Computers & security, 111*, 102490. https://doi.org/10.1016/j.cose.2021.102490

Breiman, L. (2001). Random forests. *Machine learning, 45*(1), 5-32. https://doi.org/10.1023/A:1010933404324

Chen, T., & Guestrin, C. (2016, August). Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining* (pp. 785-794). ACM. https://doi.org/10.1145/2939672.2939785

Fernando, D. W., Komninos, N., & Chen, T. (2020). A study on the evolution of ransomware detection using machine learning and deep learning techniques. *IoT, 1*(2), 551–604. https://doi.org/10.3390/iot1020030

Geurts, P., Ernst, D., & Wehenkel, L. (2006). Extremely randomized trees. *Machine learning, 63*(1), 3-42. https://doi.org/10.1007/s10994-006-6226-1

Gibert, D., Mateu, C., & Planes, J. (2020). The rise of machine learning for detection and classification of malware: Research developments, trends and challenges. *Journal of*

Network and Computer Applications, *153*, 102526. https://doi.org/10.1016/j.jnca.2019.102526

Greenwarbler. (2024). *Malware-Benign PE files* [Data set]. https://www.kaggle.com/datasets/greenwarbler/malware-benignpe-files

Grinsztajn, L., Oyallon, E., & Varoquaux, G. (2022). Why do tree-based models still outperform deep learning on typical tabular data?. *Advances in neural information processing systems, 35*, 507-520.

Hampton, N., Baig, Z., & Zeadally, S. (2018). Ransomware behavioural analysis on windows platforms. *Journal of Information Security and Applications, 40,* 44–51. https://doi.org/10.1016/j.jisa.2018.02.008

Jordaney, R., Sharad, K., Dash, S. K., Wang, Z., Papini, D., Nouretdinov, I., & Cavallaro, L. (2017). Transcend: Detecting concept drift in malware classification models. In *26th USENIX security symposium (USENIX security 17)* (pp. 625-642).

Kapoor, A., Gupta, A., Gupta, R., Tanwar, S., Sharma, G., & Davidson, I. E. (2022). Ransomware detection, avoidance, and mitigation scheme: A review and future directions. *Sustainability, 14*(1), 8. https://doi.org/10.3390/su14010008

Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., & Liu, T.-Y. (2017). LightGBM: A highly efficient gradient boosting decision tree. *Advances in Neural Information Processing Systems, 30*, 3146–3154.

Kok, S., Abdullah, A., & Jhanjhi, N. (2022). Early detection of crypto-ransomware using pre-encryption detection algorithm. *Journal of King Saud University – Computer and Information Sciences, 34*(5), 1984–1999. https://doi.org/10.1016/j.jksuci.2020.06.012

Kolawole, O. (2025). *Ransomware and goodware PE header* [Data set]. https://www.kaggle.com/datasets/oriolakolawole/ransomware-and-goodware-pe-header

Kolter, J. Z., & Maloof, M. A. (2006). Learning to detect and classify malicious executables in the wild. *Journal of Machine Learning Research*, 7, 2721–2744.

Manavi, F., & Hamzeh, A. (2021). Static detection of ransomware using LSTM network and PE header. In 2021 *26th International Computer Conference, Computer Society of Iran (CSICC)* (pp. 1–5). IEEE. https://doi.org/10.1109/CSICC52343.2021.9420580

Manavi, F., & Hamzeh, A. (2022). Ransomware detection based on PE header using convolutional neural networks. *The ISC International Journal of Information Security, 14*(2), 181–192. https://doi.org/10.22042/isecure.2021.262846.595

Meland, P. H., Bayoumy, Y. F. F., & Sindre, G. (2020). The ransomware-as-a-service economy within the darknet. *Computers & Security, 92*, 101762. https://doi.org/10.1016/j.cose.2020.101762

Moreira, C. C., Moreira, D. C., & de Sales Jr., C. S. (2023). Improving ransomware detection based on portable executable header using xception convolutional neural network. *Computers & Security, 130*, 103265. https://doi.org/10.1016/j.cose.2023.103265

Moussaileb, R., Cuppens, N., Lanet, J. L., & Bouder, H. L. (2021). A survey on windows-based ransomware taxonomy and detection mechanisms. *ACM Computing Surveys (CSUR), 54*(6), 1-36. https://doi.org/10.1145/3453153

Nataraj, L., Karthikeyan, S., Jacob, G., & Manjunath, B. S. (2011). Malware images: Visualization and automatic classification. In *Proceedings of the 8th International Symposium on Visualization for Cyber Security* (pp. 1–7). ACM. https://doi.org/10.1145/2016904.2016908

Oz, H., Aris, A., Levi, A., & Uluagac, A. S. (2022). A survey on ransomware: Evolution, taxonomy, and defense solutions. *ACM Computing Surveys, 54*(11s), 238. https://doi.org/10.1145/3514229

Pendlebury, F., Pierazzi, F., Jordaney, R., Kinder, J., & Cavallaro, L. (2019). {TESSERACT}: Eliminating experimental bias in malware classification across space and time. In *28th USENIX security symposium (USENIX Security 19)* (pp. 729-746).

Poudyal, S., Gupta, K. D., & Sen, S. (2019). PEFile analysis: A static approach to ransomware analysis. *International Journal of Forensic Computer Science, 14*(1), 34–39. https://doi.org/10.5769/J201901004

Prokhorenkova, L., Gusev, G., Vorobev, A., Dorogush, A. V., & Gulin, A. (2018). CatBoost: Unbiased boosting with categorical features. *Advances in Neural Information Processing Systems, 31*, 6638–6648.

Raff, E., Barker, J., Sylvester, J., Brandon, R., Catanzaro, B., & Nicholas, C. (2018). Malware detection by eating a whole exe. In *Proceedings of the AAAI Workshop on Artificial Intelligence for Cyber Security*. AAAI Press.

Saxe, J., & Berlin, K. (2015). Deep neural network based malware detection using two dimensional binary visualization. In *2015 10th International Conference on Malicious and Unwanted Software (MALWARE)* (pp. 11–20). IEEE. https://doi.org/10.1109/MALWARE.2015.7413680

Schultz, M. G., Eskin, E., Zadok, E., & Stolfo, S. J. (2001). Data mining methods for detection of new malicious executables. In *Proceedings 2001 IEEE Symposium on Security and Privacy* (pp. 38–49). IEEE. https://doi.org/10.1109/SECPRI.2001.924286

Schwartz, R., Dodge, J., Smith, N. A., & Etzioni, O. (2020). Green ai. *Communications of the ACM, 63*(12), 54–63. https://doi.org/10.1145/3381831

Sgandurra, D., Muñoz-González, L., Mohsen, R., & Lupu, E. C. (2016). Automated dynamic analysis of ransomware: Benefits, limitations and use for detection. *arXiv preprint arXiv:1609.03020*. https://doi.org/10.48550/arXiv.1609.03020

Shafiq, M. Z., Tabish, S. M., Mirza, F., & Farooq, M. (2009). PE-Miner: Mining structural information to detect malicious executables in realtime. In *International Workshop on Recent Advances in Intrusion Detection* (pp. 121–141). Springer. https://doi.org/10.1007/978-3-642-04342-0_7

Statista. (2025). *Total annual amount of money received by ransomware actors worldwide from 2017 to 2024*. https://www.statista.com

Strubell, E., Ganesh, A., & McCallum, A. (2019). Energy and policy considerations for deep learning in NLP. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics* (pp. 3645–3650). ACL. https://doi.org/10.18653/v1/P19-1355

Ucci, D., Aniello, L., & Baldoni, R. (2019). Survey of machine learning techniques for malware analysis. *Computers & Security, 81,* 123-147. https://doi.org/10.1016/j.cose.2018.11.001

Vidyarthi, D., Kumar, C., Rakshit, S., & Chansarkar, S. (2019). Static malware analysis to identify ransomware properties. *International Journal of Computer Science Issues, 16*(3), 10–17. https://doi.org/10.5281/zenodo.3252963

Vuran, N. (2025). *Windows PE malware dataset* [Data set]. https://www.kaggle.com/datasets/nisavuran/windows-pe-malware-dataset

Zahoora, U., Rajarajan, M., Pan, Z., & Khan, A. (2022). Zero-day ransomware attack detection using deep contractive autoencoder and voting based ensemble classifier. *Applied Intelligence, 52*(12), 13941–13960. https://doi.org/10.1007/s10489-022-03244-6

Zhang, H., Xiao, X., Mercaldo, F., Ni, S., Martinelli, F., & Sangaiah, A. K. (2019). Classification of ransomware families with machine learning based on N-gram of opcodes. *Future Generation Computer Systems, 90*, 211–221. https://doi.org/10.1016/j.future.2018.07.052